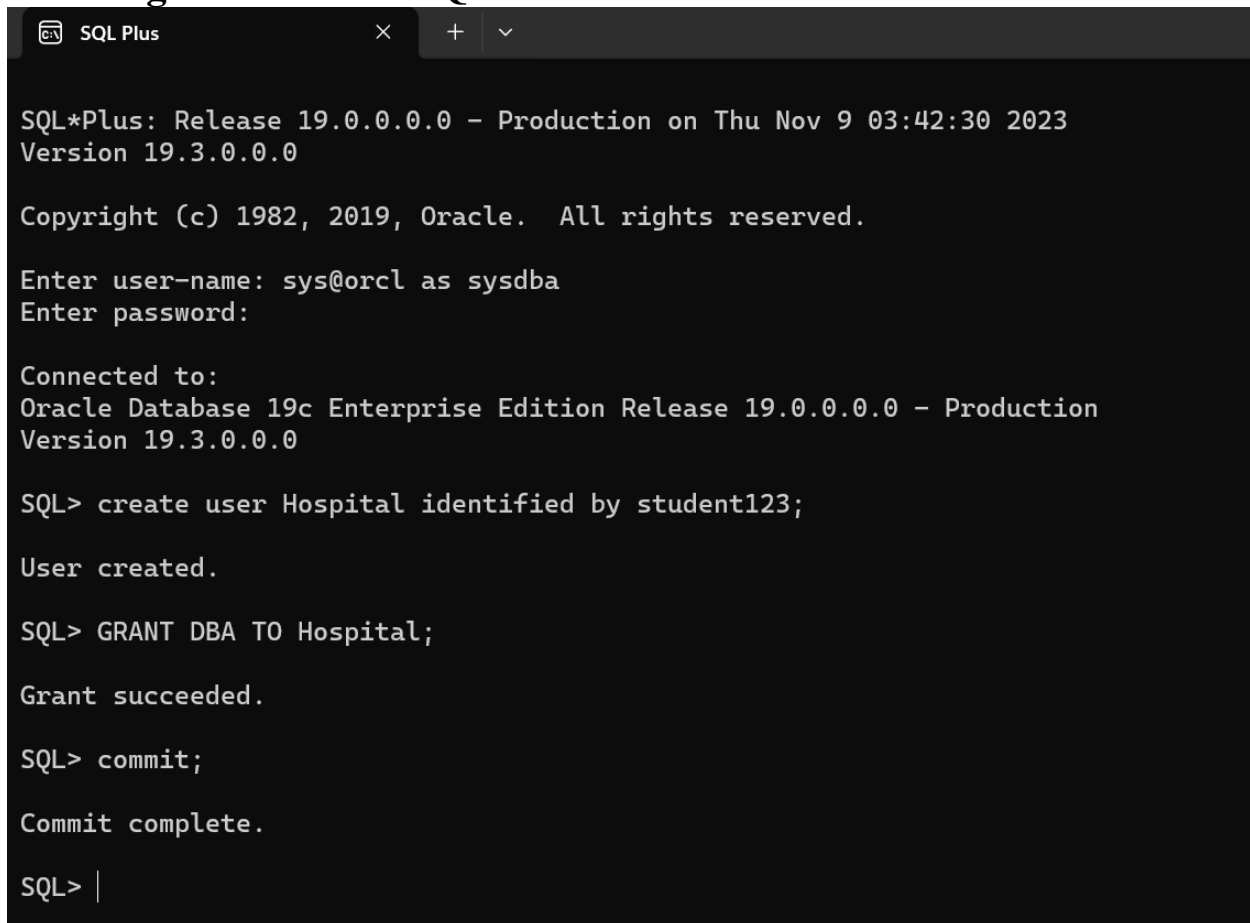


Data Modelling and Database Design

DATABASE IMPLEMENTATION



Creating Database on SQL Plus:



```
SQL Plus
SQL*Plus: Release 19.0.0.0.0 - Production on Thu Nov 9 03:42:30 2023
Version 19.3.0.0.0

Copyright (c) 1982, 2019, Oracle. All rights reserved.

Enter user-name: sys@orcl as sysdba
Enter password:

Connected to:
Oracle Database 19c Enterprise Edition Release 19.0.0.0.0 - Production
Version 19.3.0.0.0

SQL> create user Hospital identified by student123;

User created.

SQL> GRANT DBA TO Hospital;

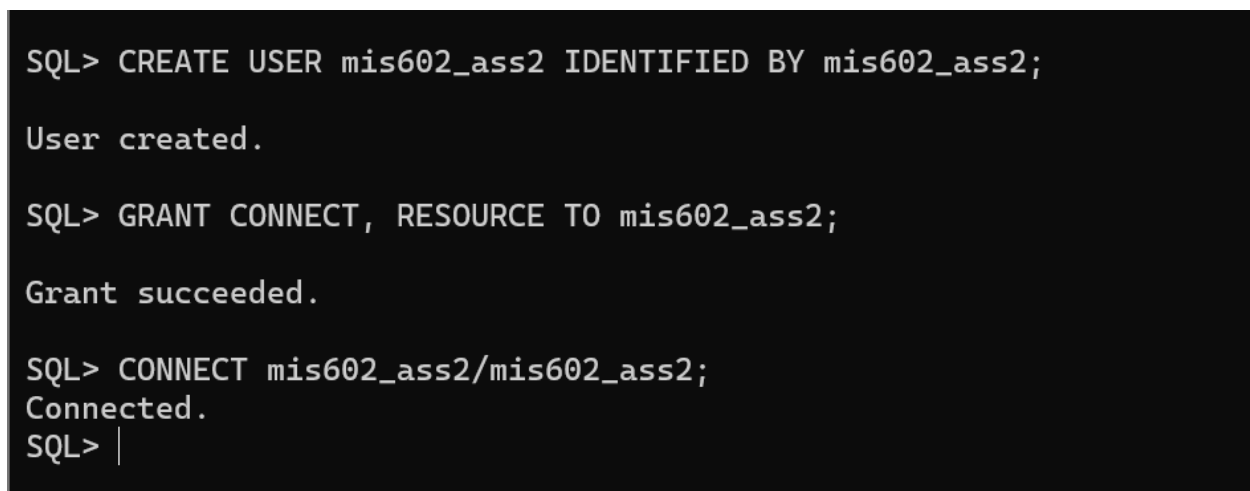
Grant succeeded.

SQL> commit;

Commit complete.

SQL> |
```

Figure 1: SQL Plus sysdba connection



```
SQL> CREATE USER mis602_ass2 IDENTIFIED BY mis602_ass2;

User created.

SQL> GRANT CONNECT, RESOURCE TO mis602_ass2;

Grant succeeded.

SQL> CONNECT mis602_ass2/mis602_ass2;
Connected.
SQL> |
```

Figure 2: new USER and DB creation mis602_ass2

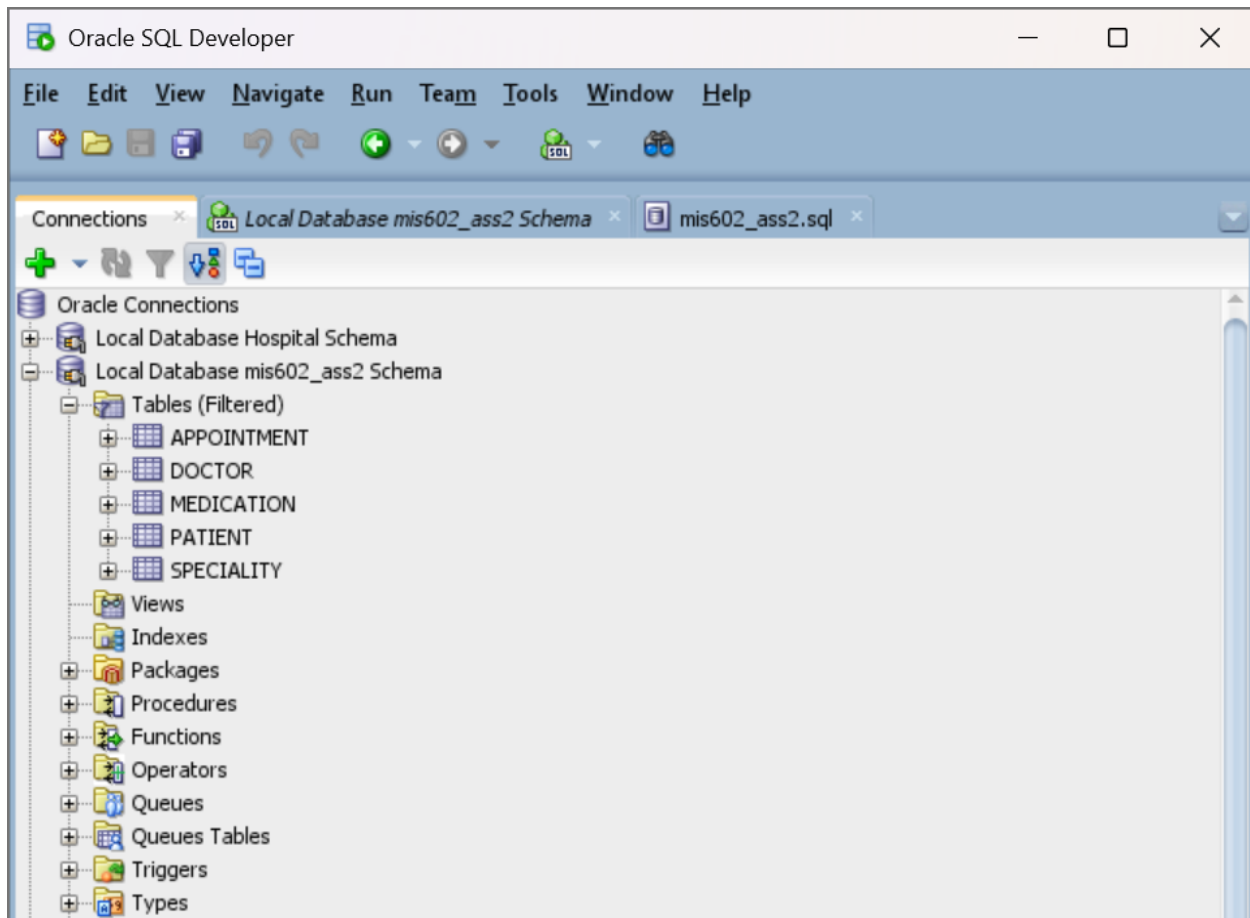
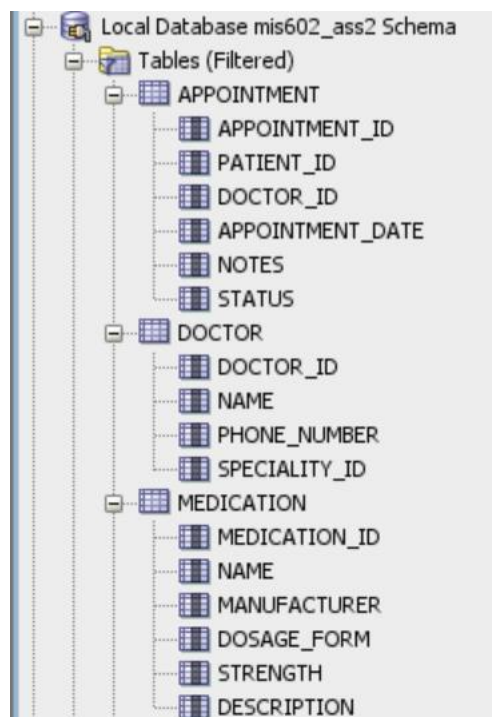


Figure 3: Oracle SQL Developer



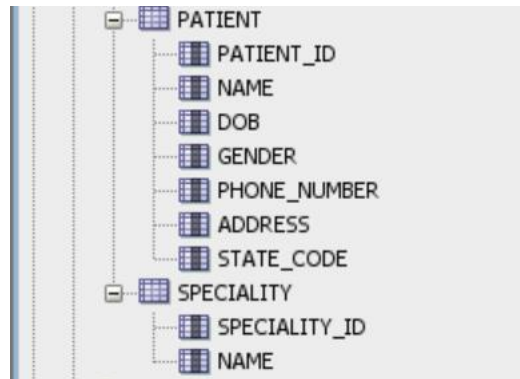
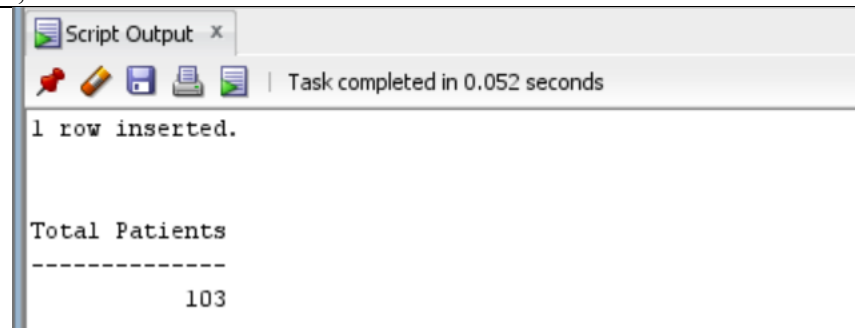


Figure 4: DB table column Rows

Questions & Solutions:

1. How many patients are there in the patient table?

```
SELECT COUNT(*) AS "Total Patients"
FROM patient;
```



2. Write a query to determine if there are more males than females in the patient table.

```
SELECT gender, COUNT(*) AS gender_count
FROM patient
GROUP BY gender;
SELECT
  (SELECT COUNT(*) FROM patient WHERE gender = 'male') AS male_count,
  (SELECT COUNT(*) FROM patient WHERE gender = 'female') AS female_count
FROM dual;
```

Script Output x	
Task completed in 0.039 seconds	
GENDER	GENDER_COUNT

male	51
female	52
MALE_COUNT	FEMALE_COUNT

51	52

- Retrieve the names of patients who have 'Lee' in their name and 'follow-up' in the appointment notes.

```
SELECT DISTINCT p.name
FROM patient AS p
JOIN appointment AS a ON p.patient_id = a.patient_id
WHERE p.name LIKE '%Lee%' AND a.notes LIKE '%follow-up%';
```

```
--Q3
SELECT DISTINCT p.name
FROM patient p
JOIN appointment a ON p.patient_id = a.patient_id
WHERE UPPER(p.name) LIKE '%LEE%'
AND UPPER(a.notes) LIKE '%FOLLOW-UP%';
```

- Retrieve the medications that have been prescribed for patients whose names ending with 'Smith'.

```
SELECT DISTINCT m.medication_name
FROM medication AS m
JOIN prescription AS p ON m.medication_id = p.medication_id
JOIN patient AS pt ON p.patient_id = pt.patient_id
WHERE pt.name LIKE '%Smith';
```

```
--Q4
SELECT DISTINCT m.name AS medication_name
FROM patient p
JOIN prescription pr ON p.patient_id = pr.patient_id
JOIN medication m ON pr.drug_id = m.medication_id
WHERE p.name LIKE '%Smith';
```

- List the doctors who have not been assigned any appointments using a subquery.

```
SELECT d.doctor_id, d.doctor_name
FROM doctor AS d
```

```
WHERE d.doctor_id NOT IN (SELECT DISTINCT doctor_id FROM appointment);
```

```
--Q5
SELECT doctor_id, doctor_name
FROM doctor
WHERE doctor_id NOT IN (SELECT DISTINCT doctor_id FROM appointment);
```

6. Retrieve the doctors who have prescribed medications containing 'pain' in their description more than 2 times.

```
SELECT d.doctor_id, d.doctor_name
FROM doctor AS d
WHERE (SELECT COUNT(*) FROM prescription AS p WHERE p.doctor_id = d.doctor_id
AND p.description LIKE '%pain%') > 2;
```

```
--Q6
SELECT d.doctor_id, d.doctor_name
FROM doctor d
JOIN prescription p ON d.doctor_id = p.doctor_id
JOIN medication m ON p.medication_id = m.medication_id
WHERE LOWER(m.description) LIKE '%pain%'
GROUP BY d.doctor_id, d.doctor_name
HAVING COUNT(m.medication_id) > 2;
```

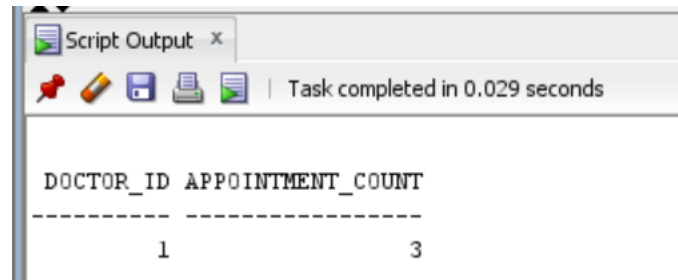
7. Write a query to find patients who have appointments scheduled in the month of June 2023.

```
SELECT DISTINCT p.name
FROM patient AS p
JOIN appointment AS a ON p.patient_id = a.patient_id
WHERE a.appointment_date >= '2023-06-01' AND a.appointment_date < '2023-07-01';
```

```
--Q7
SELECT DISTINCT p.patient_id, p.patient_name
FROM patient p
JOIN appointment a ON p.patient_id = a.patient_id
WHERE EXTRACT(MONTH FROM a.appointment_date) = 6
AND EXTRACT(YEAR FROM a.appointment_date) = 2023;
```

8. Write a query to find the doctor with the highest number of appointments scheduled in a single day.

```
SELECT doctor_id, COUNT(*) AS appointment_count
FROM appointment
GROUP BY doctor_id, TRUNC(appointment_date)
ORDER BY appointment_count DESC
FETCH FIRST 1 ROW ONLY;
```

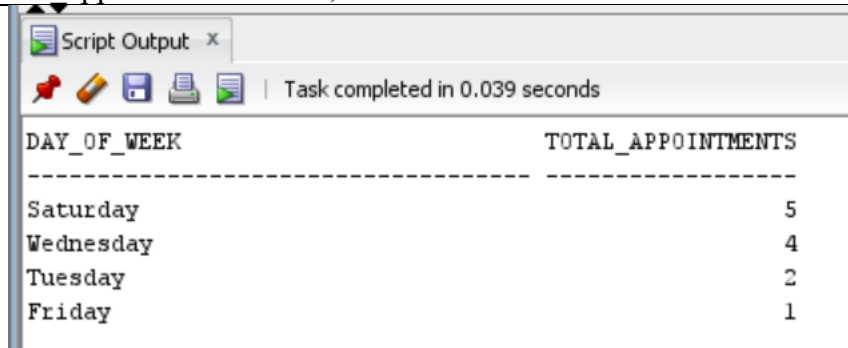


Script Output x | Task completed in 0.029 seconds

DOCTOR_ID	APPOINTMENT_COUNT
1	3

9. List the total number of appointments based on the day of the week

```
SELECT TO_CHAR(appointment_date, 'Day') AS day_of_week, COUNT(*) AS
total_appointments
FROM appointment
GROUP BY TO_CHAR(appointment_date, 'Day')
ORDER BY total_appointments DESC;
```



Script Output x | Task completed in 0.039 seconds

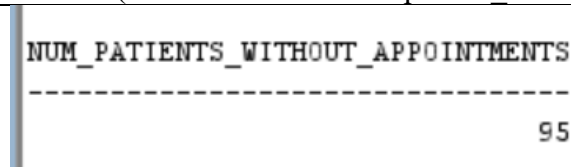
DAY_OF_WEEK	TOTAL_APPOINTMENTS
Saturday	5
Wednesday	4
Tuesday	2
Friday	1

10. Display the patients if they are sharing the same birth month as another patient.

```
SELECT DISTINCT p1.name AS patient1, p2.name AS patient2
FROM patient AS p1
JOIN patient AS p2 ON MONTH(p1.birth_date) = MONTH(p2.birth_date) AND p1.patient_id
< p2.patient_id;
```

11. How many patients have never had an appointment or scheduled any appointments so far?

```
SELECT COUNT(*) AS num_patients_without_appointments
FROM patient
WHERE patient_id NOT IN (SELECT DISTINCT patient_id FROM appointment);
```



NUM_PATIENTS_WITHOUT_APPOINTMENTS
95

12. Show medications that have never been prescribed to by any doctor using a JOIN of your choice.

```
SELECT m.medication_name
FROM medication AS m
LEFT JOIN prescription AS p ON m.medication_id = p.medication_id
WHERE p.prescription_id IS NULL;
```

13. List the doctors who have treated at least one patient from 'WA'

```
SELECT DISTINCT d.doctor_id, d.doctor_name
FROM doctor AS d
JOIN appointment AS a ON d.doctor_id = a.doctor_id
JOIN patient AS p ON a.patient_id = p.patient_id
WHERE p.state = 'WA';
```

14. Which medicine has the second-highest overall prescription rate?

```
SELECT medication_name
FROM medication
WHERE medication_id = (SELECT medication_id
                      FROM prescription
                      GROUP BY medication_id
                      ORDER BY COUNT(*) DESC
                      LIMIT 1, 1);
```

15. Retrieve the patients who have never cancelled an appointment.

```
SELECT DISTINCT p.name
FROM patient AS p
WHERE p.patient_id NOT IN (SELECT DISTINCT patient_id FROM appointment WHERE
status = 'Cancelled');
```

16. Retrieve the youngest and oddest female patient.

```
SELECT *
FROM patient
WHERE gender = 'Female'
ORDER BY birth_date DESC
LIMIT 1;
```

```
--Q16
SELECT *
FROM patient
WHERE gender = 'Female'
ORDER BY birth_date DESC, patient_id ASC
FETCH FIRST 1 ROW ONLY;
```


17. Find medications that have appeared only once in any prescription.

```
SELECT m.medication_name
FROM medication AS m
LEFT JOIN prescription AS p ON m.medication_id = p.medication_id
GROUP BY m.medication_id
HAVING COUNT(p.prescription_id) = 1;
```

18. Find medications that have appeared only once in any prescription along with the doctors who prescribed them.

```
SELECT m.medication_name, d.doctor_name
FROM medication AS m
LEFT JOIN prescription AS p ON m.medication_id = p.medication_id
JOIN doctor AS d ON p.doctor_id = d.doctor_id
GROUP BY m.medication_id
HAVING COUNT(p.prescription_id) = 1;
```

19. Retrieve the patients who have had appointments with at least three different doctors

```
SELECT p.patient_id, p.name, COUNT(DISTINCT a.doctor_id) AS unique_doctors
FROM patient AS p
JOIN appointment AS a ON p.patient_id = a.patient_id
GROUP BY p.patient_id, p.name
HAVING unique_doctors >= 3;
```

```
--Q19
SELECT p.patient_id, p.patient_name
FROM patient p
JOIN appointment a ON p.patient_id = a.patient_id
GROUP BY p.patient_id, p.patient_name
HAVING COUNT(DISTINCT a.doctor_id) >= 3;
```

20. In not more than 200 words, comment on whether the tables are in 3NF. Justify your argument with relevant examples, and then explain at least two ways to improve this database based on

The Third Normal Form (3NF) requires that a table must be in the Second Normal Form (2NF) and, additionally, all non-prime attributes (attributes not part of any candidate key) must be functionally dependent on the primary key. Based on the provided information, we can evaluate the tables in the database:

1. Patient Table:

- It seems to be in 3NF as long as patient information is fully dependent on the patient_id (primary key).

- To improve, ensure that there are no partial dependencies, and all attributes are directly related to the primary key.

2. Doctor Table:

- Similarly, the Doctor Table seems to be in 3NF with a primary key of doctor_id.
- Ensure that attributes are fully dependent on the primary key.

3. Medication Table:

- Appears to be in 3NF as it contains medication information solely dependent on the primary key, medication_id.
- Verify that there are no partial dependencies.

4. Appointment Table:

- It might be in 3NF if there is a primary key, but this is not provided.
- To improve, ensure that the table has a primary key and that all attributes are functionally dependent on it.

5. Prescription Table:

- Similar to the Appointment Table, it should have a primary key defined for 3NF.
- Make sure all attributes depend on the primary key.